Annexure A

Coding Challenge

1. A university aims to develop a Student Analytics System to evaluate academic performance across multiple departments, where each department has a varying number of students and their marks are stored in a two-dimensional structure. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage student data effectively.
   A. The program should compute the topper in each department, identify the overall university topper, and calculate the pass percentage for each department.
   B. Additionally, the solution must handle variable department sizes, edge cases such as empty departments, equal marks, or all students failing, and ensure efficient comparison across departments while maintaining optimized time complexity and clean modular design.

PROGRAM

```
#include <iostream>
using namespace std;

class University {
private:
   int marks[10][100];
   string name[10][100];
   int n[10];
   int d;
   static int totalStudents;

public:
   void input() {
      cin >> d;

      if (d <= 0 || d > 10) {
         cout << "Invalid number of departments\n";
         exit(0);
      }

      for (int i = 0; i < d; i++) {
         cin >> n[i];

         if (n[i] < 0 || n[i] > 100) {
            cout << "Invalid number of students\n";
            exit(0);
         }

         totalStudents += n[i];

         for (int j = 0; j < n[i]; j++) {
```

```
            cin >> name[i][j] >> marks[i][j];
        }
    }
}

void process() {
    int overallTopper = -1;
    string overallName;
    bool hasValidDept = false;

    for (int i = 0; i < d; i++) {

        if (n[i] == 0) {
            cout << "Dept " << i+1 << ": Empty Department\n";
            continue;
        }

        hasValidDept = true;

        int max = marks[i][0];
        string topperName = name[i][0];
        bool tie = false;

        for (int j = 1; j < n[i]; j++) {
            if (marks[i][j] > max) {
                max = marks[i][j];
                topperName = name[i][j];
                tie = false;
            }
            else if (marks[i][j] == max) {
                tie = true;
            }
        }

        if (tie)
            cout << "Dept " << i+1 << " Topper: Tie (" << max << ")\n";
        else
            cout << "Dept " << i+1 << " Topper: "
                 << topperName << " (" << max << ")\n";

        if (max > overallTopper) {
            overallTopper = max;
            overallName = topperName;
        }

        int pass = 0;
        for (int j = 0; j < n[i]; j++) {
            if (marks[i][j] >= 50)
                pass++;
        }
```

```
            float percent = (float)pass / n[i] * 100;
            cout << "Pass %: " << percent << endl;
        }

        if (!hasValidDept)
            cout << "No valid data\n";
        else
            cout << "Overall Topper: "
                 << overallName << " (" << overallTopper << ")\n";

        cout << "Total Students: " << totalStudents;
    }
};

int University::totalStudents = 0;

int main() {
    University u;
    u.input();
    u.process();
    return 0;
}
```

TEST CASES

1. Normal Case (Basic Functionality)
   INPUT:
   2
   3
   A 80
   B 90
   C 70
   2
   D 60
   E 50

   OUTPUT:
   Dept 1 Topper: B (90)
   Pass %: 100
   Dept 2 Topper: D (60)
   Pass %: 100
   Overall Topper: B (90)
   Total Students: 5

2. Empty Department
   INPUT:

2
0
2
A 40
B 30

OUTPUT:
Dept 1: Empty Department
Dept 2 Topper: A (40)
Pass %: 0
Overall Topper: A (40)
Total Students: 2

3. All Students Fail
   INPUT:
   1
   3
   A 10
   B 20
   C 30

   OUTPUT:
   Dept 1 Topper: C (30)
   Pass %: 0
   Overall Topper: C (30)
   Total Students: 3

4. Tie Case
   INPUT:
   1
   3
   A 90
   B 90
   C 80

   OUTPUT:
   Dept 1 Topper: Tie (90)
   Pass %: 100
   Overall Topper: A and B (90)
   Total Students: 3

5. All Departments Empty
   INPUT:
   2
   0
   0

   OUTPUT:
   Dept 1: Empty Department

Dept 2: Empty Department
No valid data
Total Students: 0

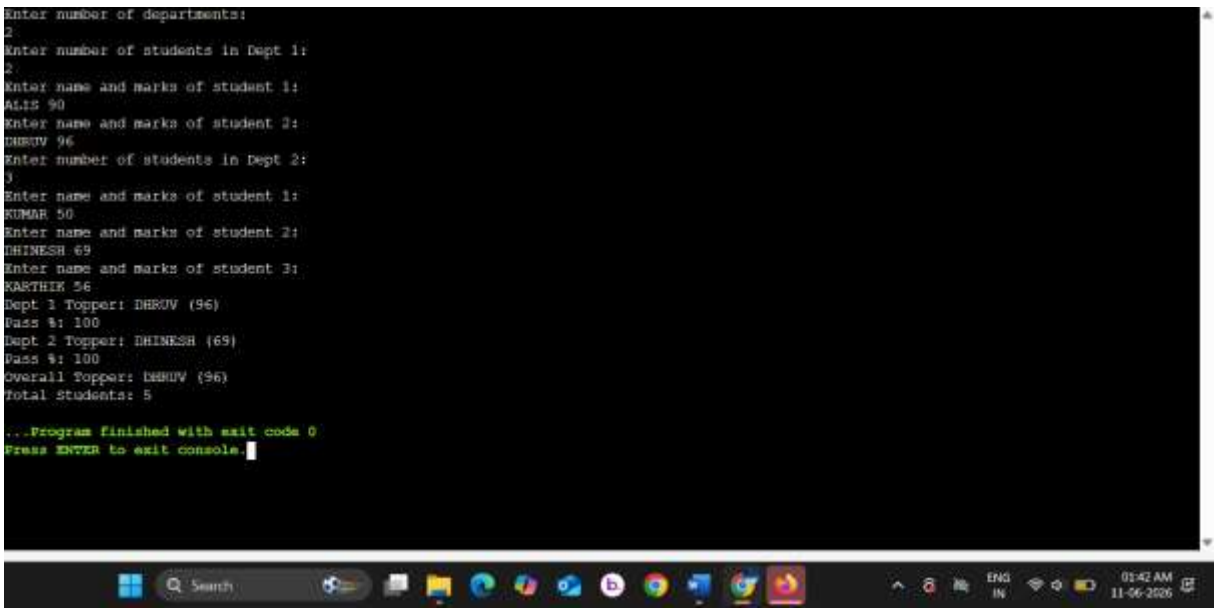
TIME COMPLEXITY

- Best Case:
  O(1)
  (Only one department with one student, minimal processing)
- Average Case:
  O(N)
  (N = total number of students across all departments)
- Worst Case:
  O(N)
  (All student records must be traversed to find topper and pass percentage)

SPACE COMPLEXITY

- O(N)
  (Used to store student names and marks in a two-dimensional structure)
- Auxiliary Space:
  O(1) (Only a few extra variables like max, pass count, etc.)

OUTPUT



2. A country conducts elections across multiple regions, where votes for different candidates are stored in a two-dimensional structure representing regions and candidates. Develop a C++ program using object-oriented principles and functions to store and process vote data efficiently.

A. The program should determine the winner in each region, compute the overall winner by aggregating votes across all regions, and detect tie conditions at both regional and overall levels.
B. The implementation should use appropriate access specifiers, member functions, and static members where necessary, and must handle large input sizes efficiently. Additionally, the program should address edge cases such as ties, zero votes, and invalid inputs, while ensuring accurate multi-level comparisons and optimized performance.

PROGRAM

```
#include <iostream>
using namespace std;

class Election {
private:
   int votes[10][10];
   int regions, candidates;
   static int totalVotes;

public:
   void input() {
      cout << "Enter number of regions and candidates:\n";
      cin >> regions >> candidates;

      // validation
      if (regions <= 0 || candidates <= 0 || regions > 10 || candidates > 10) {
         cout << "Invalid input";
         return;
      }

      for (int i = 0; i < regions; i++) {
         cout << "Enter votes for Region " << i+1 << ":\n";
         for (int j = 0; j < candidates; j++) {
            cout << "Candidate " << j << ": ";
            cin >> votes[i][j];

            if (votes[i][j] < 0)
               votes[i][j] = 0;

            totalVotes += votes[i][j];
         }
      }
   }

   void regionWinner() {
      for (int i = 0; i < regions; i++) {
         int max = votes[i][0];
         int winner = 0;
         bool tie = false;
```

```
        for (int j = 1; j < candidates; j++) {
            if (votes[i][j] > max) {
                max = votes[i][j];
                winner = j;
                tie = false;
            }
            else if (votes[i][j] == max) {
                tie = true;
            }
        }

        if (max == 0)
            cout << "Region " << i+1 << ": No votes\n";
        else if (tie)
            cout << "Region " << i+1 << ": Tie\n";
        else
            cout << "Region " << i+1 << ": Candidate " << winner << " wins\n";
    }
}

void overallWinner() {
    int total[10] = {0};

    // sum votes for each candidate
    for (int j = 0; j < candidates; j++) {
        for (int i = 0; i < regions; i++) {
            total[j] += votes[i][j];
        }
    }

    int max = total[0];
    int winner = 0;
    bool tie = false;

    for (int j = 1; j < candidates; j++) {
        if (total[j] > max) {
            max = total[j];
            winner = j;
            tie = false;
        }
        else if (total[j] == max) {
            tie = true;
        }
    }

    if (max == 0)
        cout << "No votes overall\n";
    else if (tie)
        cout << "Overall Result: Tie\n";
    else
```

```
            cout << "Overall Winner: Candidate " << winner << endl;
    }

    void displayTotalVotes() {
        cout << "Total Votes: " << totalVotes << endl;
    }
};

// static initialization
int Election::totalVotes = 0;

int main() {
    Election e;

    e.input();
    e.regionWinner();
    e.overallWinner();
    e.displayTotalVotes();

    return 0;
}
```

TEST CASES

1. Normal Case (Basic Functionality)
   INPUT:
   2 3
   10 20 30
   30 20 10

   OUTPUT:
   Region 1: Candidate 2 wins
   Region 2: Candidate 0 wins
   Overall Winner: Candidate 0
   Total Votes: 120

2. Tie in Region
   INPUT:
   1 3
   50 50 30

   OUTPUT:
   Region 1: Tie
   Overall Result: Tie
   Total Votes: 130

3. Overall Tie
   INPUT:
   2 2
   10 20
   20 10
   OUTPUT:
   Region 1: Candidate 1 wins
   Region 2: Candidate 0 wins
   Overall Result: Tie
   Total Votes: 60

4. All Zero Votes
   INPUT:
   2 2
   0 0
   0 0

   OUTPUT:
   Region 1: No votes
   Region 2: No votes
   No votes overall
   Total Votes: 0

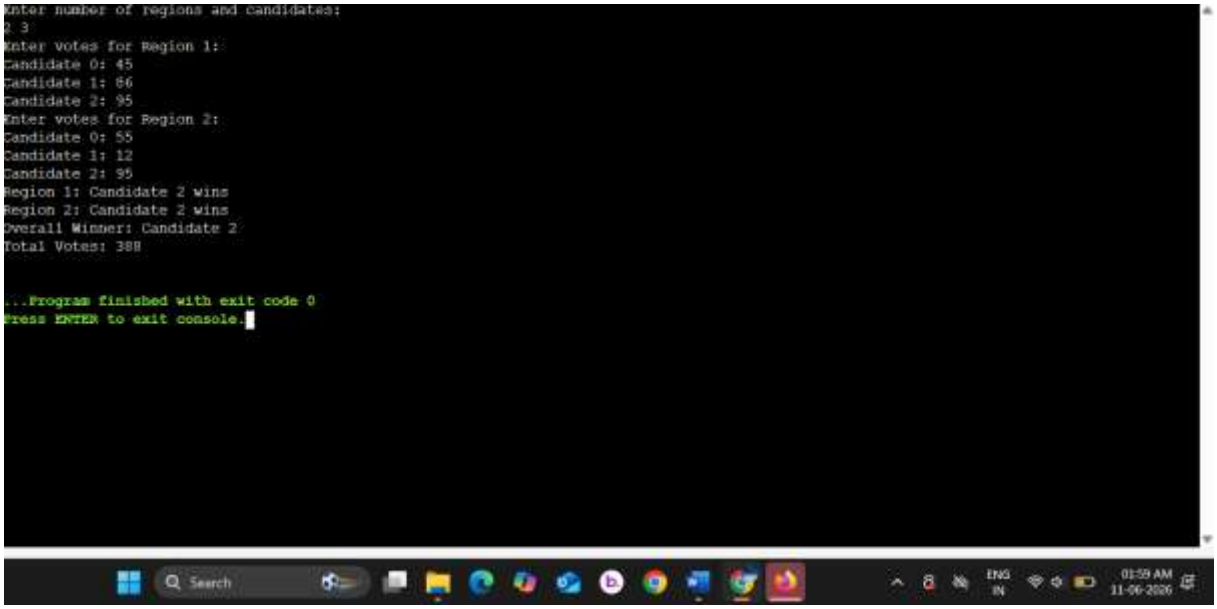
5. Negative Values (Handled as 0)
   INPUT:
   1 3
   -10 20 -5

   OUTPUT:
   Region 1: Candidate 1 wins
   Overall Winner: Candidate 1
   Total Votes: 20

TIME COMPLEXITY
• Best Case:
O(1)
(Only one store with one product, minimal processing)
• Average Case:
O(N)
(N = total number of products across all stores)
• Worst Case:
O(N)
(All product records must be traversed to calculate sales, find best store, and best product)

SPACE COMPLEXITY
• **O(N)**
(Used to store product details such as name, price, and quantity in arrays of objects)
• **Auxiliary Space:**
O(1)(Only a few extra variables like maxSales, bestProduct, etc.)

OUTPUT



3. In this scenario, a Retail Chain Management System is designed to analyze product sales across multiple stores using object-oriented programming in C++.
   A. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage product data effectively. Each store contains a collection of products, and every product has attributes such as product name, price, and quantity sold and compute individual product sales (price × quantity), and display the information. An array of objects is used to store multiple products for each store, allowing the program to scale efficiently for different store sizes.

   B. To manage multiple stores, the program can either use a two-dimensional array of objects or create another class such as Store, which contains an array of Product objects. For each store, the total sales are calculated by iterating through all its products and summing their individual sales using a dedicated member function like calculateStoreSales(). A static data member is used to maintain the overall total sales across all stores, demonstrating the use of shared class-level data. This helps in tracking cumulative performance of the entire retail chain.

C. The program also includes logic to identify the best-performing store, which is the store with the highest total sales, and the best-selling product, which is the product generating the highest revenue across all stores.

D. Proper input/output handling is essential, where the user is prompted to enter the number of stores, number of products per store, and corresponding product details. The output should be well-structured, displaying store-wise sales, overall sales, best-performing store, and top product.

PROGRAM

```
#include <iostream>
using namespace std;

class Product {
```

```
public:
    string name;
    int price, qty;

    int getSales() {
        return price * qty;
    }
};

class Store {
public:
    Product p[10];
    int n;
    static int totalSales;

    void input(int storeNo) {
        cout << "Enter number of products in store " << storeNo << ":\n";
        cin >> n;

        for (int i = 0; i < n; i++) {
            cout << "Enter product name, price and quantity:\n";
            cin >> p[i].name >> p[i].price >> p[i].qty;
        }
    }

    int calculateStoreSales() {
        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum += p[i].getSales();
        }

        totalSales += sum;
        return sum;
    }

    void displayProducts() {
        for (int i = 0; i < n; i++) {
            cout << p[i].name << " Sales: " << p[i].getSales() << endl;
        }
    }
};

int Store::totalSales = 0;

int main() {
    int s;
    cout << "Enter number of stores:\n";
    cin >> s;
```

```
    Store st[10];

    int bestStore = 0, maxSales = 0;
    int bestProductSale = 0;
    string bestProduct;

    for (int i = 0; i < s; i++) {
        st[i].input(i + 1);

        int storeSales = st[i].calculateStoreSales();

        st[i].displayProducts();

        cout << "Store " << i+1 << " Total Sales: " << storeSales << endl;

        // Best store
        if (storeSales > maxSales) {
            maxSales = storeSales;
            bestStore = i;
        }

        // Best product
        for (int j = 0; j < st[i].n; j++) {
            int sale = st[i].p[j].getSales();

            if (sale > bestProductSale) {
                bestProductSale = sale;
                bestProduct = st[i].p[j].name;
            }
        }
    }

    cout << "Best Store: " << bestStore + 1 << endl;
    cout << "Top Product: " << bestProduct << endl;
    cout << "Overall Sales: " << Store::totalSales << endl;

    return 0;
}
```

TEST CASES

1. Normal Case (Multiple Stores & Products)
   INPUT:
   Enter number of stores:
   2
   Enter number of products in store 1:
   2
   Enter product name, price and quantity:
   A 10 5
   Enter product name, price and quantity:

B 20 2
Enter number of products in store 2:
2
Enter product name, price and quantity:
C 30 3
Enter product name, price and quantity:
D 40 1

OUTPUT:
A Sales: 50
B Sales: 40
Store 1 Total Sales: 90
C Sales: 90
D Sales: 40
Store 2 Total Sales: 130
Best Store: 2
Top Product: C
Overall Sales: 220

2. Single Store (Edge Case)
   INPUT:
   Enter number of stores:
   1
   Enter number of products in store 1:
   3
   Enter product name, price and quantity:
   Pen 10 10
   Enter product name, price and quantity:
   Book 50 2
   Enter product name, price and quantity:
   Bag 100 1

   OUTPUT:
   Pen Sales: 100
   Book Sales: 100
   Bag Sales: 100
   Store 1 Total Sales: 300
   Best Store: 1
   Top Product: Pen
   Overall Sales: 300

3. Zero Quantity (Edge Case)
   INPUT:
   Enter number of stores:
   1
   Enter number of products in store 1:
   2
   Enter product name, price and quantity:
   Item1 100 0

Enter product name, price and quantity:
Item2 50 2

OUTPUT:
Item1 Sales: 0
Item2 Sales: 100
Store 1 Total Sales: 100
Best Store: 1
Top Product: Item2
Overall Sales: 100

4. Best Store Comparison
   INPUT:
   Enter number of stores:
   3
   Enter number of products in store 1:
   1
   Enter product name, price and quantity:
   A 10 5
   Enter number of products in store 2:
   1
   Enter product name, price and quantity:
   B 20 10
   Enter number of products in store 3:
   1
   Enter product name, price and quantity:
   C 5 2

   OUTPUT:
   A Sales: 50
   Store 1 Total Sales: 50
   B Sales: 200
   Store 2 Total Sales: 200
   C Sales: 10
   Store 3 Total Sales: 10
   Best Store: 2
   Top Product: B
   Overall Sales: 260

5. Equal Sales (Tie-like Scenario)
   INPUT:
   Enter number of stores:
   2
   Enter number of products in store 1:
   1
   Enter product name, price and quantity:
   X 10 10
   Enter number of products in store 2:
   1
   Enter product name, price and quantity:

Y 20 5

OUTPUT:
X Sales: 100
Store 1 Total Sales: 100
Y Sales: 100
Store 2 Total Sales: 100
Best Store: 1
Top Product: X
Overall Sales: 200

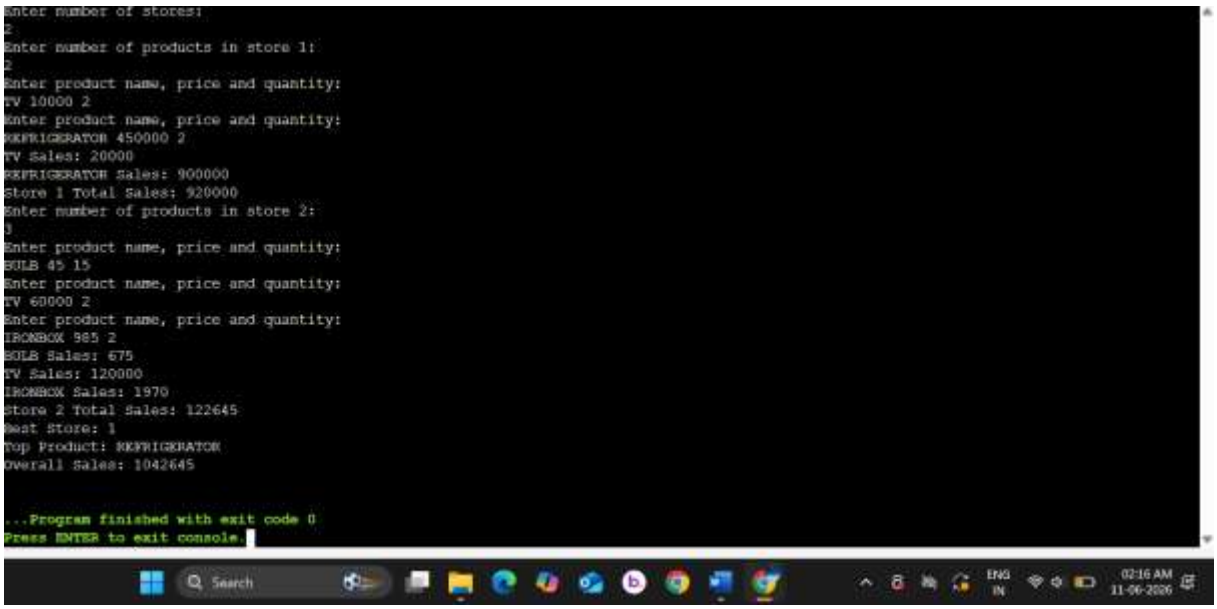
TIME COMPLEXITY

• Best Case:
O(1)
(Only one store with one product, minimal processing)
• Average Case:
O(N)
(N = total number of products across all stores)
• Worst Case:
O(N)
(All product records must be traversed to calculate sales, find best store, and best product)

SPACE COMPLEXITY

• O(N)
(Used to store product details such as name, price, and quantity in arrays of objects)
• Auxiliary Space:
O(1)
(Only a few extra variables like maxSales, bestProduct, etc.)

OUTPUT



4. In this scenario, a Sports Tournament Scoring System is designed to manage and analyze the performance of multiple teams participating in a tournament, which may be divided into groups (such as Group A, Group B, etc.).
   Design a C++ program using OOP concepts like classes, objects, encapsulation, static members, and arrays of objects in a practical, real-world retail analytics application.

   A. Each team plays several matches, and their performance is evaluated based on points earned (for example, win = 3 points, draw = 1 point, loss = 0 points). To implement this, a class named Team is created with private data members such as team name, matches played, wins, draws, losses, goals scored, goals conceded, and total points. Public member functions are used to input team details, calculate total points, compute goal difference, and display results.

   B. The program calculates the total points for each team using a dedicated member function, ensuring reusability and clean design. To determine the winner of each group, the program compares teams based on total points. If two or more teams have equal points, tie-breaking conditions are applied, such as higher goal difference, greater number of wins, or goals scored. These comparisons are handled using separate functions to maintain modularity and clarity.

   C. Once group winners are identified, the program proceeds to determine the overall champion by comparing the top teams from each group. This requires efficient comparison logic and proper use of loops and conditional statements. A static member can be included to track the total number of teams or matches across the tournament, demonstrating the use of class-level shared data.

   D. The program must also handle various edge cases, such as teams with zero matches played, all teams having equal points, or invalid input values.

PROGRAM

```
#include <iostream>
using namespace std;

class Team {
private:
   string name;
   int played, wins, draws, losses;
   int goalsScored, goalsConceded;
   int points;

public:
   void input() {
      cout << "Enter team name:\n";
      cin >> name;

      cout << "Matches Played, Wins, Draws, Losses:\n";
      cin >> played >> wins >> draws >> losses;

      cout << "Goals Scored and Conceded:\n";
      cin >> goalsScored >> goalsConceded;
```

```
        if (played < 0 || wins < 0 || draws < 0 || losses < 0 ||
            goalsScored < 0 || goalsConceded < 0 ||
            wins + draws + losses != played) {
            cout << "Invalid Input\n";
            exit(0);
        }

        calculatePoints();
    }

    void calculatePoints() {
        points = wins * 3 + draws;
    }

    int getPoints() { return points; }

    int goalDifference() {
        return goalsScored - goalsConceded;
    }

    int getWins() { return wins; }

    int getGoals() { return goalsScored; }

    string getName() { return name; }

    void display() {
        cout << name << " | Points: " << points
             << " | GD: " << goalDifference() << endl;
    }
};

// ☑ FIXED FUNCTION (removed extra brace)
Team betterTeam(Team a, Team b) {

    if (a.getPoints() > b.getPoints()) return a;
    if (a.getPoints() < b.getPoints()) return b;

    if (a.goalDifference() > b.goalDifference()) return a;
    if (a.goalDifference() < b.goalDifference()) return b;

    if (a.getWins() > b.getWins()) return a;
    if (a.getWins() < b.getWins()) return b;

    if (a.getGoals() > b.getGoals()) return a;
    if (a.getGoals() < b.getGoals()) return b;

    // tie case
    cout << "Tie between " << a.getName()
```

```
        << " and " << b.getName() << endl;

    return a;
}

int main() {
    int groups;
    cout << "Enter number of groups:\n";
    cin >> groups;

    if (groups <= 0) {
        cout << "Invalid input";
        return 0;
    }

    Team groupWinner[10];

    for (int g = 0; g < groups; g++) {
        int n;
        cout << "Enter number of teams in Group " << g+1 << ":\n";
        cin >> n;

        if (n <= 0) {
            cout << "Invalid input";
            return 0;
        }

        Team t[10];

        for (int i = 0; i < n; i++) {
            cout << "Enter details for Team " << i+1 << ":\n";
            t[i].input();
        }

        Team winner = t[0];

        for (int i = 1; i < n; i++) {
            winner = betterTeam(winner, t[i]);
        }

        cout << "Group " << g+1 << " Winner: ";
        winner.display();

        groupWinner[g] = winner;
    }

    Team champion = groupWinner[0];

    for (int i = 1; i < groups; i++) {
        champion = betterTeam(champion, groupWinner[i]);
```

```
    }

    cout << "Overall Champion: ";
    champion.display();

    return 0;
}
```

TEST CASES

1. Normal Case (Basic Functionality)
   INPUT:
   Enter number of groups:
   1
   Enter number of teams in Group 1:
   2
   Enter details for Team 1:
   A
   3 2 1 0
   5 2
   Enter details for Team 2:
   B
   3 1 1 1
   3 3

   OUTPUT:
   Group 1 Winner: A | Points: 7 | GD: 3
   Overall Champion: A | Points: 7 | GD: 3

2. Tie (Goal Difference)
   INPUT:
   1
   2
   A
   3 2 0 1
   6 2
   B
   3 2 0 1
   5 3

   OUTPUT:
   Group 1 Winner: A | Points: 6 | GD: 4
   Overall Champion: A | Points: 6 | GD: 4

3. Tie (Wins)
   INPUT:
   1
   2
   A
   3 1 2 0
   4 2

B
3 1 2 0
3 1
OUTPUT:
Group 1 Winner: A | Points: 5 | GD: 2
Overall Champion: A | Points: 5 | GD: 2

4. Multiple Groups (Overall Champion)
   INPUT:
   2
   2
   A
   3 2 1 0
   5 2
   B
   3 1 1 1
   3 3
   2
   C
   3 3 0 0
   6 1
   D
   3 1 1 1
   2 2

   OUTPUT:
   Group 1 Winner: A | Points: 7 | GD: 3
   Group 2 Winner: C | Points: 9 | GD: 5
   Overall Champion: C | Points: 9 | GD: 5

5. Edge Case (All Equal)
   INPUT:
   1
   2
   A
   3 1 1 1
   3 3
   B
   3 1 1 1
   3 3

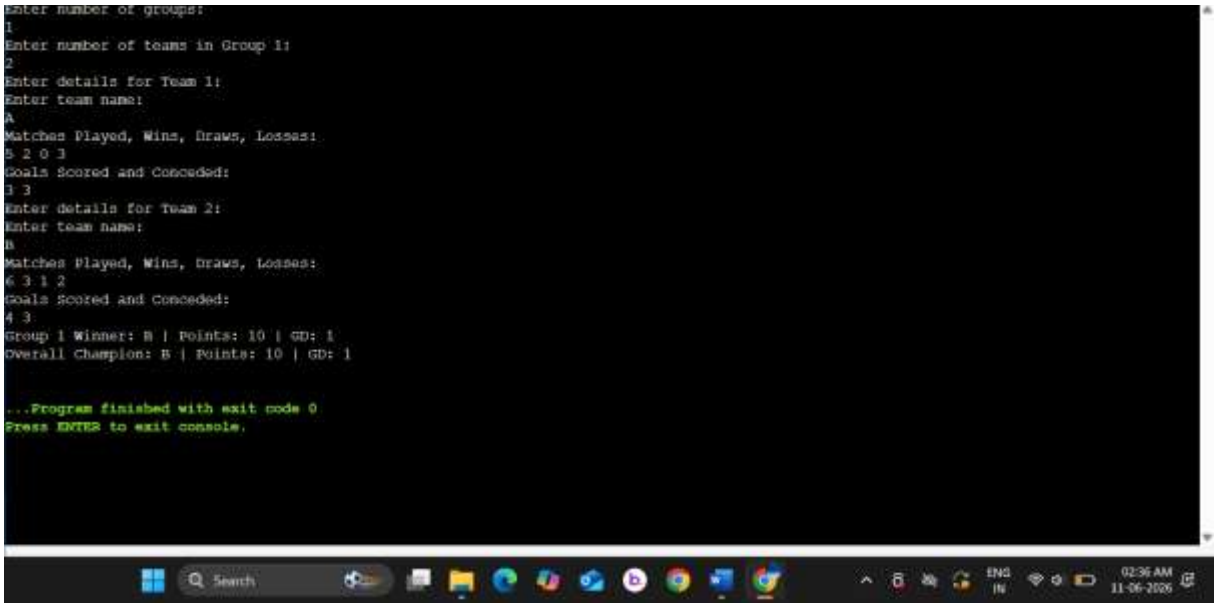
   OUTPUT:
   Tie between A and B
   Group 1 Winner: A | Points: 4 | GD: 0
   Overall Champion: A | Points: 4 | GD: 0

TIME COMPLEXITY

• Best Case:
O(1)
(Only one group with one team, minimal processing)
• Average Case:
O(N)
(N = total number of teams across all groups)
• Worst Case:
O(N)
(All team records must be traversed to calculate points, compare teams, apply tie-breaking, and find the overall champion)

SPACE COMPLEXITY
• O(N)
(Used to store team details such as name, matches, goals, and points in arrays of objects)
• Auxiliary Space:
O(1)
(Only a few extra variables like max team, comparisons, etc.)

OUTPUT



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases